

-0.5cm -0.3cm



Traitement et Analyse de Données

Preprocessing - Cours 3 : Transformation et Encodage

Dr Seydou Nourou SYLLA

Data Scientist Senior
Departement TIC
Universite Alioune Diop de Bambey

January 26, 2026



Overview

- 1. Encodage des variables catégorielles**
- 2. Enrichissement des données**
- 3. Augmentation des données**
- 4. Méthodes Flags**
- 5. Transformation et Normalisation**
- 6. Réduction de dimension**
- 7. Conclusion**



Encodage des variables catégorielles



Pourquoi encoder les variables ?

- Les algorithmes de machine learning nécessitent souvent des données numériques.
- Variables catégorielles doivent être converties en format compréhensible.



Techniques d'encodage

- **Encodage simple (Label Encoding) :**
 - Conversion des catégories en valeurs entières.
- **One-Hot Encoding :**
 - Création de colonnes binaires pour chaque catégorie.
- **Encodage ordinal :**
 - Utilisé lorsque les catégories ont un ordre naturel.

Exemple - Label Encoding

Transformation

Couleur	Couleur Encodée
rouge	0
bleu	1
vert	2
rouge	0
bleu	1

Exemple - One-Hot Encoding

Transformation

Ville	ville_Lyon	ville_Marseille	ville_Paris
Paris	0	0	1
Lyon	1	0	0
Marseille	0	1	0
Paris	0	0	1

Exemple - Encodage Ordinal

Transformation (ordre préservé : Faible < Moyen < Élevé)

Satisfaction	Satisfaction Encodée
Faible	0
Moyen	1
Élevé	2
Moyen	1
Faible	0



Binary Encoding

- **Principe** : Combiner les avantages du label encoding et du one-hot encoding.
- Étapes :
 - Convertir les catégories en valeurs entières.
 - Convertir ces valeurs en binaire.
 - Créer des colonnes pour chaque bit.
- Avantages : Réduction de la dimensionnalité.

Exemple – Binary Encoding

Transformation étape par étape

Catégorie	Label	Bit 0	Bit 1	Bit 2
A	0	0	0	0
B	1	1	0	0
C	2	0	1	0
D	3	1	1	0

Avantage

Pour 4 catégories, One-Hot nécessite 4 colonnes, Binary Encoding seulement 3 colonnes (réduction de dimensionnalité).



Frequency Encoding

- **Principe** : Encoder les catégories en fonction de leur fréquence d'apparition dans les données.
- Avantages : Simple et efficace.
- Limitation : Ne capture pas la relation entre les catégories.

Exemple – Frequency Encoding

Transformation (Fréquences : Paris=3, Lyon=2, Marseille=1)

Ville	Ville Fréquence
Paris	3
Lyon	2
Paris	3
Marseille	1
Lyon	2
Paris	3



Target Encoding

- **Principe** : Remplacer chaque catégorie par la moyenne de la variable cible pour cette catégorie.
- Utilisation :
 - Très utilisé pour les modèles supervisés.
- Risques : Overfitting si mal utilisé.

Exemple - Target Encoding

Transformation (Moyennes : Paris=110, Lyon=82.5, Marseille=90)

Ville	Prix	Ville Target
Paris	100	110
Lyon	80	82.5
Paris	120	110
Marseille	90	90
Lyon	85	82.5



Hashing Encoding

- **Principe** : Appliquer une fonction de hachage pour convertir les catégories en indices numériques.
- Avantages :
 - Réduction de la dimension.
 - Pas besoin de connaître les catégories à l'avance.
- Limitation : Collisions possibles.

Exemple - Hashing Encoding

Transformation ($n_features=4$, collision possible : Nice et Paris = 2)

Ville	Hash Index
Paris	2
Lyon	1
Marseille	0
Toulouse	3
Nice	2



Comparaison des techniques d'encodage

- **Critères de choix :**
 - Dimensionnalité des données résultantes.
 - Relation entre les catégories et la variable cible.
 - Compatibilité avec l'algorithme utilisé.
- Exemple :
 - One-hot encoding : Bon pour des catégories indépendantes.
 - Target encoding : Meilleur pour des relations complexes.

Enrichissement des donnees



Introduction

Objectif : Comprendre les techniques d'enrichissement et d'augmentation des données pour améliorer les performances des modèles d'apprentissage automatique.

- Qu'est-ce que l'enrichissement et l'augmentation des données ?
- Importance dans le Machine Learning.
- Différences clés entre enrichissement et augmentation.



Enrichissement des données



Définition de l'enrichissement

- **Principe** : Ajouter des informations supplémentaires pour rendre les données plus exploitables.
- Sources d'enrichissement :
 - Données externes (API, bases de données ouvertes).
 - Calcul de nouvelles variables (features engineering).

Techniques d'enrichissement

- **Création de variables dérivées :**
 - Variables temporelles : Jour de la semaine, saison, etc.
 - Variables géographiques : Distance, altitude, etc.
- **Fusion de données :**
 - Jointures entre différentes bases.
- **Ajout de données tierces :**
 - Météo, indicateurs économiques, etc.



Augmentation des données



Définition de l'augmentation

- **Principe** : Générer de nouvelles données à partir des données existantes pour augmenter la diversité.
- Utilisation : Réduction du surapprentissage (overfitting).



Techniques d'augmentation pour les images

- **Transformations géométriques :**
 - Rotation, mise à l'échelle, translation.
- **Modification de l'intensité :**
 - Changement de luminosité, contraste, etc.
- **Techniques avancées :**
 - GANs (Generative Adversarial Networks).
 - Mixup et CutMix.



Techniques d'augmentation pour le texte

- **Remplacement synonyme :**
 - Utilisation de WordNet ou autres ressources lexicales.
- **Insertion ou suppression :**
 - Ajout ou suppression aléatoire de mots.
- **Traductions croisées :**
 - Traduire dans une autre langue puis revenir à la langue source.



Techniques d'augmentation pour les séries temporelles

- **Jittering :**
 - Ajout de bruit aléatoire.
- **Warping temporel :**
 - Étirement ou compression de segments temporels.
- **Découpage et réassemblage :**
 - Perturber l'ordre tout en conservant la structure globale.



Méthodes Flags



Introduction aux méthodes Flags

- **Objectif** : Identifier et annoter les données ayant des caractéristiques spécifiques.
- Utilisation : Détection de valeurs aberrantes, échantillonnage.



Techniques Flags

- **Flags de qualité :**
 - Indiquer les données manquantes ou suspectes.
- **Flags de segmentation :**
 - Annoter les données par catégorie ou classe.
- **Flags pour outliers :**
 - Basé sur des méthodes statistiques ou de machine learning.



Applications des Flags

- Pré-traitement avant modélisation.
- Surveillance des processus en temps réel.
- Visualisation et reporting.

Exemple - Flags de qualité

Transformation

ID	Âge	Flag Manquant	Flag Suspect
1	25	0	0
2	NaN	1	0
3	150	0	1
4	30	0	0
5	NaN	1	0

Flag Manquant : 1 si valeur manquante, 0 sinon

Flag Suspect : 1 si valeur hors domaine (ex: âge > 120), 0 sinon

Exemple - Flags de segmentation

Transformation

Client	Revenu	Flag Segment A	Flag Segment B
C1	50000	1	0
C2	120000	0	1
C3	45000	1	0
C4	150000	0	1
C5	55000	1	0

Segment A : Revenu < 60000, Segment B : Revenu $\geq$ 60000

Exemple – Flags pour outliers

Transformation (détection par IQR)

ID	Valeur	Flag Outlier	Z-score
1	25	0	0.5
2	28	0	0.8
3	150	1	4.2
4	26	0	0.6
5	200	1	5.8

Flag Outlier : 1 si $|Z - score| > 3$ ou hors IQR, 0 sinon



Transformation et Normalisation



Pourquoi transformer les données ?

- Réduction de la variance des échelles des variables.
- Préparer les données pour certains algorithmes sensibles (SVM, KNN).
- Accélérer la convergence des modèles d'optimisation.

Normalisation des données

- **Principe** : Ramener les valeurs des variables dans un intervalle spécifique (souvent $[0, 1]$).
- Méthodes courantes :
 - **Min-Max Scaling** :

$$X' = \frac{X - \min(X)}{\max(X) - \min(X)}$$

- Applications :
 - Modèles utilisant des distances (KNN, clustering).

Standardisation des données

- **Principe** : Transformer les données pour qu'elles aient une moyenne de 0 et un écart-type de 1.

- Formule :

$$X' = \frac{X - \mu}{\sigma}$$

- Avantages :
 - Convient aux modèles linéaires et aux réseaux neuronaux.



Transformation logarithmique

- **Principe** : Appliquer une transformation logarithmique pour réduire l'impact des valeurs extrêmes.
- Formule :

$$X' = \log(X + 1)$$

- Applications :
 - Données financières, données fortement asymétriques.

Transformation par racine carrée

- **Principe** : Réduire l'amplitude des grandes valeurs tout en conservant leur distribution.
- Formule :

$$X' = \sqrt{X}$$

- Applications :
 - Données avec des écarts-types élevés.

Transformation par Box-Cox

- **Principe** : Trouver une transformation optimale pour rendre les données plus normales.
- Formule générale :

$$X'(\lambda) = \begin{cases} \frac{X^\lambda - 1}{\lambda}, & \text{si } \lambda \neq 0 \\ \log(X), & \text{si } \lambda = 0 \end{cases}$$

- Applications :
 - Données asymétriques ou non gaussiennes.



Réduction de dimension



Pourquoi réduire la dimension ?

- Simplifier les données et réduire la complexité.
- Éliminer les corrélations inutiles.
- Accélérer l'apprentissage des modèles.



Méthodes principales

- **PCA (Analyse en Composantes Principales)**
- **Sélection de caractéristiques :**
 - Basée sur l'importance des variables (arbre de décision, random forests).



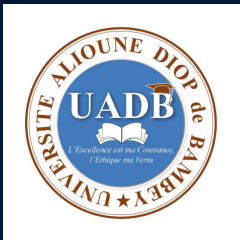
Conclusion



Conclusion

- Le prétraitement des données est une étape cruciale pour garantir le succès des modèles d'apprentissage.
- Choisir la bonne méthode dépend des données et des objectifs.
- Étapes clés :
 - Gestion des données manquantes.
 - Transformation et normalisation.
 - Encodage et réduction de dimension.

Questions ?



Thank you for your attention

Dr Seydou Nourou SYLLA

Data Scientist Senior
Departement TIC
Universite Alioune Diop de Bambey

January 26, 2026

Code Python - Label Encoding (1/2)

Code Python

```
1 import pandas as pd
2 import numpy as np
3
4 # Données d'exemple
5 df = pd.DataFrame({
6     'couleur': ['rouge', 'bleu', 'vert', 'rouge', 'bleu']
7 })
8
9 # Méthode 1: Utiliser factorize()
10 df['couleur_encoded'], unique_labels = pd.factorize(df['couleur'])
11 print(df)
```

Code Python - Label Encoding (2/2)

Code Python

```
1 # Méthode 2: Utiliser astype('category').cat.codes
2 df['couleur_encoded2'] = df['couleur'].astype('category').cat.codes
3 print(df)
```

Code Python - One-Hot Encoding

Code Python

```
1 import pandas as pd
2
3 # Données d'exemple
4 df = pd.DataFrame({
5     'ville': ['Paris', 'Lyon', 'Marseille', 'Paris']
6 })
7
8 # One-Hot Encoding avec get_dummies
9 df_encoded = pd.get_dummies(df['ville'], prefix='ville')
10 df_final = pd.concat([df, df_encoded], axis=1)
11 print(df_final)
12
13 # Avec drop_first=True pour éviter la multicollinéarité
14 df_encoded = pd.get_dummies(df['ville'], prefix='ville', drop_first=True)
```

Code Python – Encodage Ordinal (1/2)

Code Python

```
1 import pandas as pd
2
3 # Données avec ordre naturel
4 df = pd.DataFrame({
5     'satisfaction': ['Faible', 'Moyen', 'Élevé', 'Moyen', 'Faible']
6 })
7
8 # Définir l'ordre
9 satisfaction_order = ['Faible', 'Moyen', 'Élevé']
10 satisfaction_map = {val: idx for idx, val in enumerate(satisfaction_order)}
11
12 # Encodage ordinal
13 df['satisfaction_encoded'] = df['satisfaction'].map(satisfaction_map)
14 print(df)
```

Code Python – Encodage Ordinal (2/2)

Code Python

```
1 # Alternative avec astype('category')
2 df['satisfaction_cat'] = pd.Categorical(df['satisfaction'],
3                                       categories=satisfaction_order,
4                                       ordered=True)
5 df['satisfaction_encoded2'] = df['satisfaction_cat'].cat.codes
```

Code Python – Binary Encoding

Code Python

```
1 import pandas as pd
2 import numpy as np
3
4 # Données d'exemple
5 df = pd.DataFrame({
6     'categorie': ['A', 'B', 'C', 'D', 'A']
7 })
8
9 # Étape 1: Label encoding
10 df['label'] = pd.factorize(df['categorie'])[0]
11
12 # Étape 2: Convertir en binaire
13 max_bits = int(np.ceil(np.log2(df['label'].max() + 1)))
14 for i in range(max_bits):
15     df[f'bit_{i}'] = (df['label'] >> i) & 1
16
17 print(df[['categorie', 'label', 'bit_0', 'bit_1', 'bit_2']])
```

Code Python – Frequency Encoding

Code Python

```
1 import pandas as pd
2
3 # Données d'exemple
4 df = pd.DataFrame({
5     'ville': ['Paris', 'Lyon', 'Paris', 'Marseille', 'Lyon', 'Paris']
6 })
7
8 # Calculer les fréquences
9 freq_map = df['ville'].value_counts().to_dict()
10
11 # Encoder avec les fréquences
12 df['ville_freq'] = df['ville'].map(freq_map)
13 print(df)
14
15 # Alternative: Utiliser la fréquence relative
16 freq_relative = df['ville'].value_counts(normalize=True).to_dict()
17 df['ville_freq_rel'] = df['ville'].map(freq_relative)
```

Code Python – Target Encoding (1/2)

Code Python

```
1 import pandas as pd
2
3 # Données d'exemple avec variable cible
4 df = pd.DataFrame({
5     'ville': ['Paris', 'Lyon', 'Paris', 'Marseille', 'Lyon'],
6     'prix': [100, 80, 120, 90, 85]
7 })
8
9 # Calculer la moyenne du prix par ville
10 target_mean = df.groupby('ville')['prix'].mean().to_dict()
11
12 # Encoder avec la moyenne de la cible
13 df['ville_target_encoded'] = df['ville'].map(target_mean)
14 print(df)
```

Code Python – Target Encoding (2/2)

Code Python

```
1 # Avec lissage (smoothing) pour éviter l'overfitting
2 global_mean = df['prix'].mean()
3 smoothing = 0.1
4 df['ville_smoothed'] = df['ville'].map(
5     lambda x: (target_mean[x] * (1 - smoothing) +
6               global_mean * smoothing)
7 )
```

Code Python - Hashing Encoding (1/2)

Code Python

```
1 import pandas as pd
2 import hashlib
3
4 # Données d'exemple
5 df = pd.DataFrame({
6     'ville': ['Paris', 'Lyon', 'Marseille', 'Toulouse', 'Nice']
7 })
8
9 # Hashing Encoding avec nombre de features
10 n_features = 4 # Nombre de colonnes à créer
11
12 def hash_encode(value, n_features):
13     # Utiliser hash() et prendre modulo
14     return abs(hash(str(value))) % n_features
15
16 # Encoder chaque catégorie
17 UADB['ville_hash'] = df['ville'].apply(lambda x: hash_encode(x, n_features))
```

Code Python - Hashing Encoding (2/2)

Code Python

```
1 # Alternative: Créer plusieurs colonnes binaires
2 for i in range(n_features):
3     df[f'hash_{i}'] = (df['ville_hash'] == i).astype(int)
```

Code Python – Flags de qualité

Code Python

```
1 import pandas as pd
2 import numpy as np
3
4 # Données d'exemple
5 df = pd.DataFrame({
6     'age': [25, np.nan, 150, 30, np.nan],
7     'salaire': [5000, 6000, 7000, np.nan, 5500]
8 })
9
10 # Flag pour valeurs manquantes
11 df['flag_age_missing'] = df['age'].isnull().astype(int)
12 df['flag_salaire_missing'] = df['salaire'].isnull().astype(int)
13
14 # Flag pour valeurs suspectes (hors domaine)
15 df['flag_age_suspect'] = ((df['age'] < 0) | (df['age'] > 120)).astype(int)
16 df['flag_salaire_suspect'] = (df['salaire'] < 0).astype(int)
```

Code Python - Flags de segmentation

Code Python

```
1 import pandas as pd
2
3 # Données d'exemple
4 df = pd.DataFrame({
5     'client': ['C1', 'C2', 'C3', 'C4', 'C5'],
6     'revenu': [50000, 120000, 45000, 150000, 55000]
7 })
8
9 # Segmentation par seuil
10 seuil = 60000
11 df['flag_segment_A'] = (df['revenu'] < seuil).astype(int)
12 df['flag_segment_B'] = (df['revenu'] >= seuil).astype(int)
13
14 # Segmentation multiple (3 segments)
15 df['flag_segment_low'] = (df['revenu'] < 50000).astype(int)
16 df['flag_segment_medium'] = ((df['revenu'] >= 50000) &
17                               (df['revenu'] < 100000)).astype(int)
```

Code Python – Flags pour outliers (1/2)

Code Python

```
1 import pandas as pd
2 import numpy as np
3 from scipy import stats
4
5 # Données d'exemple
6 df = pd.DataFrame({
7     'valeur': [25, 28, 150, 26, 200, 27, 29]
8 })
9
10 # Méthode 1: Z-score
11 z_scores = np.abs(stats.zscore(df['valeur']))
12 df['flag_outlier_zscore'] = (z_scores > 3).astype(int)
13
14 # Méthode 2: IQR (Interquartile Range)
15 Q1 = df['valeur'].quantile(0.25)
16 Q3 = df['valeur'].quantile(0.75)
17 IQR = Q3 - Q1
18 lower_bound = Q1 - 1.5 * IQR
19 upper_bound = Q3 + 1.5 * IQR
```

Code Python – Flags pour outliers (2/2)

Code Python

```
1 # Combiner les flags
2 df['flag_outlier'] = ((df['flag_outlier_zscore'] == 1) |
3                       (df['flag_outlier_iqr'] == 1)).astype(int)
4
5 # Ajouter les z-scores pour information
6 df['z_score'] = z_scores
7
8 print(df[['valeur', 'z_score', 'flag_outlier_zscore',
9           'flag_outlier_iqr', 'flag_outlier']])
```